PROJECT REPORT

Autonomous Robots

MAPPING, PERCEPTION, PATH PLANNING AND NAVIGATION

Submitted by Group C

Sathwik Nagasundra Sharma

Siddhant Tiwari

Zicheng Cai

Nishith Kumar Alugandula

Course

Autonomous Robots

Ostbayerische Technische Hochschule Amberg-Weiden

Department of Electrical Engineering, Media and Computer Science

Abstract

This report documents the design and implementation of an autonomous navigation controller for a RosBot robot in the Webots simulation Worlds. The robot must first reach the blue pillar and then the yellow pillar while avoiding green hazardous ground, normal obstacles, too-narrow passages and floating wall structures. The solution uses only onboard IMU sensors. The controller is organised into hardware and odometry, perception, occupancy-grid mapping, global A* planning, local Dynamic Window Approach motion control, reactive safety, replanning and mission-level state control.

1 Project Objective

The goal of the project is to implement autonomous navigation for a RosBot in Webots Mazes. The robot starts in an unknown maze and must complete the mission in the following order:

1. build a useful map using onboard sensors,
2. detect and localise the blue and yellow pillars,
3. navigate from the start area to the blue pillar,
4. after reaching blue, navigate to the yellow pillar,
5. avoid the green ground region and all physical obstacles.

The implementation is designed for dynamic mazes , so the robot should work even when it is placed at a different initial pose. The robot does not assume a predefined map and does not use Webots Supervisor position data.

2 Project Structure

The codebase is divided into clear layers. Each file has one main responsibility, which makes the controller easier to debug and extend.

File	Main role
`main.py`	Creates robot, perception, navigator and mission objects.
`my_robot.py`	Device setup wrapper, odometry, coordinate conversion and low-level motion.
`setup.py`	Enables motors, sensors, cameras, lidar and IMU devices.
`perception.py`	Interprets RGB, depth and lidar data.
`map.py`	Maintains the occupancy grid, frontiers and path-planning map.
`astar_2_spline.py`	Runs A* search and smooths the final path.
`navigation.py`	Executes plan–follow–replan navigation with safety checks.
`mission.py`	Controls initial scan, exploration and final blue-to-yellow traversal.
`CONSTANTS.py`	Stores tunable parameters and map marker values.
`utils.py`	Provides helper functions such as colour segmentation and Bresenham lines.

Table 1: Main files used in the autonomous robot controller.

3 System Architecture

The controller follows a layered architecture. The hardware layer reads sensors and controls the motors. The perception layer converts raw camera and lidar data into useful information such as coloured pillars, green hazards and depth obstacles. The map layer stores this information in an occupancy grid. The navigation layer plans and follows paths, while the mission layer decides which phase the robot is currently executing.

Layer	Responsibility
Hardware / pose	Sensor access, wheel motors, encoder odometry and compass heading.
Perception	HSV colour detection, pillar localisation, green-floor detection and depth-obstacle projection.
Mapping	Log-odds occupancy grid, special cell marking, frontier detection and obstacle inflation.
Planning	Global A* path search with wall-clearance penalty and B-spline smoothing.
Navigation	Waypoint following, Dynamic Window Approach, emergency checks and replanning.
Mission	Initial scan, exploration, pillar discovery and final traversal.

Table 2: Layered system architecture.

4 Occupancy Grid Mapping

The map is stored as a grid of cell values. Normal obstacles, unknown cells, free space, pillars, green hazards and floating walls have different values so that they can be visualised and handled differently during planning.

For each lidar endpoint, a Bresenham line is drawn from the robot cell to the hit cell. Cells before the endpoint are updated as free space, while the final cell is updated as occupied. Instead of trusting one single sensor reading, the map uses a probabilistic occupancy grid approach with log-odds values. This helps reduce the effect of noisy lidar readings. If a false obstacle is detected only once, later free-space readings can reduce its confidence again. If the same cell is detected as occupied many times, the confidence increases and the cell becomes a stronger obstacle.

This probabilistic update makes the occupancy grid more stable. It reduces the effect of temporary noisy obstacle detections, but still allows real walls and objects to become strong obstacles after multiple sensor observations.

Special cells such as green carpet and floating walls are protected. This means that later lidar readings cannot accidentally erase hazards or depth-detected obstacles. This is necessary because green ground and floating walls are detected using camera/depth information, while the horizontal lidar may miss them in later scans.

5 Perception

The perception module converts sensor readings into semantic information. It uses HSV colour segmentation because colour thresholds are easier to tune in HSV than in RGB.

Pillar detection

Blue and yellow pillars are detected by thresholding their HSV colour masks. If the coloured blob has enough pixels, the object is accepted as a pillar. The horizontal centroid gives the bearing and the median depth over the blob gives the range.

$$x_r = r\cos(\beta), \tag{1}$$

$$y_r = r\sin(\beta) \tag{2}$$

Here r is the median depth and β is the bearing from the image centre. The local point is transformed into the world frame using the robot pose and then stored as a map coordinate.

Green hazard detection

The green area is a forbidden region. The robot checks the lower central part of the RGB image for green pixels and uses depth to verify that the green region is close. Detected green pixels are projected to the ground plane and stamped into the map as `GREEN_CARPET`. During navigation these cells are treated as hard obstacles.

Depth obstacle detection

A horizontal lidar can miss objects that are too low, flat on the floor or floating above the scan plane. To handle this, the depth camera is used as an additional obstacle-detection source. Before using the depth data, very close points are removed using a minimum-depth constant, because values too close to the sensor can be noisy or unreliable. In the same way, points that are too far away are also ignored using a maximum-depth constant, because distant depth measurements are less accurate and may create false obstacles in the map.

After this filtering, the remaining reliable depth points are projected into the ground plane. Ground-level obstacles are marked as normal obstacles, while elevated obstacles are marked separately as floating walls. This helps the planner avoid objects that the horizontal lidar alone may not detect.

6 Exploration Strategy

At the start, the robot has no complete map. The mission first performs a slow in-place scan. The 360-degree turn is divided into small slices, and the robot waits for the sensors to settle before adding lidar data to the grid. This produces a cleaner initial map.

After that, the robot uses frontier exploration. A frontier is a free cell next to an unknown cell. Frontier clusters represent useful places to explore because they lie at the boundary of known and unknown space.

During exploration, the robot also keeps checking for the blue and yellow pillars using the camera and depth data. When a pillar is detected with enough confidence, its position is stored. The pillars may be discovered in any order, but their saved positions are later used for the final mission sequence.

The robot continues selecting reachable frontier centroids until both pillars are localised. After both pillar positions are known, the exploration phase ends. The robot then starts the final traversal: it first moves to the blue pillar and, after reaching it, plans and moves from the blue pillar to the yellow pillar.

7 A* Path Planning

Global path planning is performed using A* search on the occupancy grid. The planner uses eight-connected motion with two-pixel steps. Straight moves are cheaper than diagonal moves, and the heuristic is based on Euclidean distance.

$$f(n) = g(n) + h(n) \tag{3}$$

The planner also adds a clearance penalty near walls. This makes the robot prefer routes that are slightly longer but safer, instead of driving

directly beside obstacles.

After the path is found, B-spline smoothing is applied. The smoothed path is easier for the local controller to follow because it reduces sharp grid-based turns.

8 Navigation and Replanning

The navigation module is the executor. It receives a goal cell from the mission, asks the planner for a path, converts the dense path into waypoints, faces the first waypoint, and then follows the path.

This plan–follow–replan strategy is important because the map changes while the robot moves. If a new obstacle, green patch or floating wall is detected, the current path becomes invalid and a new path must be planned.

9 DW Approach

The local motion controller uses a Dynamic Window Approach style method. Several linear and angular velocity candidates are sampled. Each candidate predicts a short future motion and receives a score based on heading alignment, distance to the target and speed. The best candidate is converted into left and right wheel velocities.

$$v_l = \frac{v - \omega L/2}{R}, \tag{4}$$

$$v_r = \frac{v + \omega L/2}{R} \tag{5}$$

Here v is the linear velocity, ω is angular velocity, L is axle length and R is wheel radius.

10 Reactive Safety

Reactive safety is checked during every navigation tick. The navigator stops and requests replanning when a dangerous condition is detected.

- green ground is close in front of the robot,
- lidar detects an obstacle inside the front cone,
- range sensors detect a near obstacle,
- depth camera detects an obstacle at collision height,

When the robot is stuck, the map can stamp the blocked region in front of the robot. The next A* search then avoids that same location. Recovery mainly uses in-place rotation and reversing, so the robot changes viewpoint and tries a new path safely.

11 Simulation Results

The mission timing was measured in two stages: from the start of the simulation until the blue pillar was reached, and from the blue pillar until the yellow pillar was reached. A demonstration video playlist is available at this link.

Maze	**Start–Blue**	**Blue–Yellow**	**Total**
Maze 1	01:43	00:49	02:32
Maze 2	01:22	00:58	02:20
Maze 3	01:10	00:37	01:47
Maze 4	01:08	01:03	02:11
Maze 5	00:48	00:29	01:17

Table 3: Simulation timing results for the autonomous robot mission.

12 Special Approaches

The same basic pipeline is used in all mazes: initial scanning, frontier exploration, pillar localisation, A* planning, DWA-based local motion and reactive replanning. The maze-wise changes are not separate hardcoded maps.Instead, small tuning changes are used to handle different maze behaviours such as noisy obstacle readings, narrow passages, close pillar detection, recovery problems and longer traversal paths.

Approach 1: Conservative reference behaviour

In this approach, the robot uses a more conservative mapping and navigation style. The map does not immediately trust a single obstacle reading. Instead, repeated obstacle detections increase confidence, while free-space readings reduce confidence. This makes the robot more stable when lidar readings contain small noise.

The navigation also follows the path more closely by using denser waypoint tracking. More replanning chances are allowed so that the robot can recover if a path becomes blocked during exploration. Pillar localisation is also made stricter, so weak or partial camera detections do not replace a better stored pillar position.

This approach is useful when the maze contains short turns, noisy map regions or places where one wrong obstacle reading could disturb the path.

Approach 2: Common baseline controller

This approach uses the normal controller without strong maze-specific tuning. It combines log-odds occupancy-grid mapping, frontier exploration, clearance-aware A* planning, B-spline path smoothing, DWA local motion and standard reactive safety checks.

This balanced setup is useful when the maze does not need special recovery or very strict arrival behaviour. It keeps the robot general and avoids depending on a specific maze layout.

Approach 3: Accurate pillar reaching and stronger recovery

This approach focuses on reaching the coloured pillars more accurately. The robot only considers the pillar reached when it is closer than the threshold defined.

The recovery behaviour is also made stronger. If the robot gets stuck in a narrow or blocked area, it creates more space for recovery and marks the blocked region in front of the robot. After this, the next A* search avoids the same failed area and tries a different route.

This approach is useful for mazes where the robot can get trapped in narrow passages or where the final pillar position must be reached more precisely.

Approach 4: Close-pillar filtering and careful local marking

This approach handles the problem of unstable close-up pillar detection. When the robot

is already very close to a known pillar, the controller avoids updating the stored pillar position again. This prevents noisy close-range depth readings from corrupting a good pillar estimate.

The blocked-area marking is also made more careful. Instead of marking a large area as closed, the robot marks only a small region when it gets stuck. This is safer in tight maze sections because it avoids accidentally blocking a valid narrow passage. Turning is also made smoother to reduce overshoot during alignment and recovery.

This approach is useful when the maze has tight spaces near the pillars or when close-range camera/depth readings can become unreliable.

Approach 5: Faster traversal with looser tolerances

This approach is tuned for faster movement through longer routes. The robot uses more confident movement and does not spend too much time correcting very small path errors. Waypoint and goal tolerances are made looser so that the robot can continue moving smoothly instead of oscillating around small target points.

The controller also allows longer navigation runs before failure, which is important for larger or more complex mazes. Depth filtering is made stricter so that very near or noisy depth points do not create false obstacles in the map.

This approach is useful for longer mazes where speed and smooth progress are more important than very fine path correction at every waypoint.

Overall, the robot follows the same autonomous method in every maze. The special approaches only adjust the behaviour of mapping, safety, recovery, turning, pillar detection and arrival checking according to the difficulty of the maze.

13 Conclusion

This project implements a complete autonomous navigation pipeline for a RosBot robot in Webots. The system builds an occupancy grid, detects blue and yellow pillars, marks green hazardous ground, explores unknown space using frontiers, plans global paths using clearance-aware A*, follows paths using local motion control and replans whenever the environment becomes unsafe. The modular design keeps perception, mapping, planning, navigation and mission logic separate, which makes the solution easier to understand, test and improve.